

Production Search Typeahead System

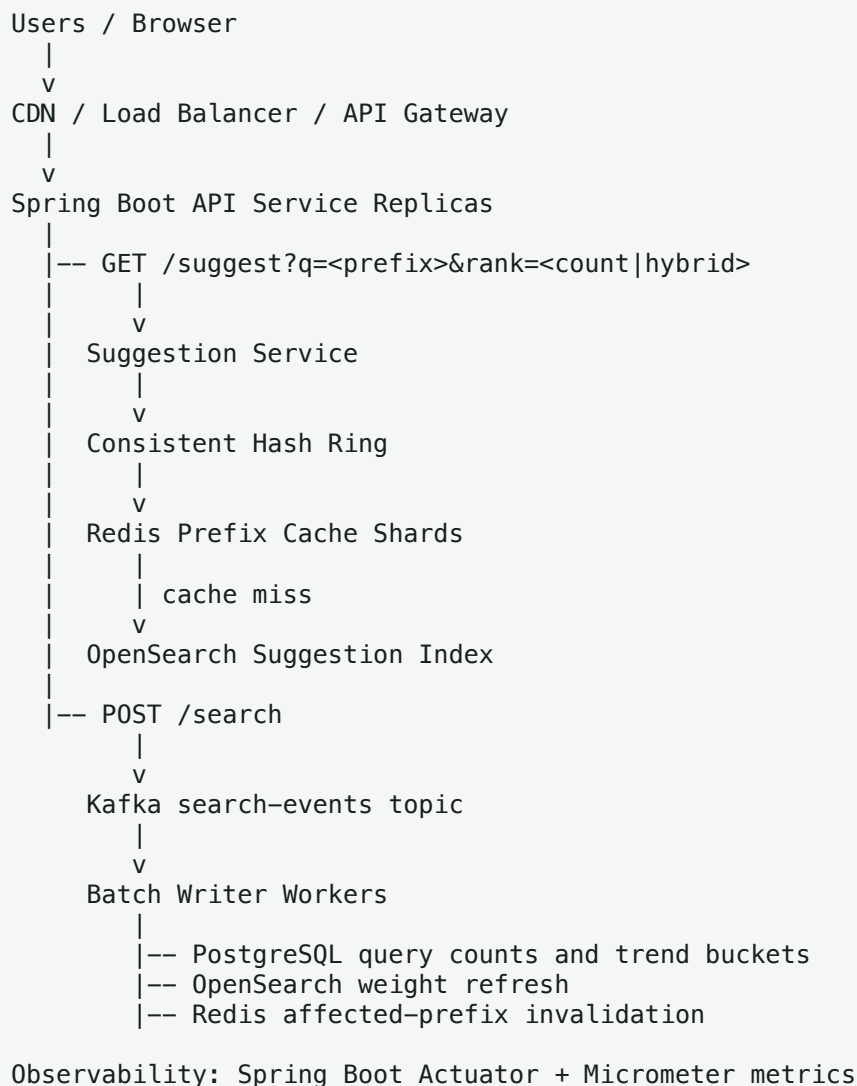
Student: Aditya Kumar Rai

SST Email: aditya.24bcs10178@sst.scaler.com

Stack: Java 21, Spring Boot, PostgreSQL, OpenSearch, Redis, Kafka

1. Architecture Diagram And Explanation

This is the high-level system architecture, not a repository or package diagram. The design separates low-latency read traffic from durable write processing so typeahead requests stay fast while query counts and trending signals are updated reliably.



The default local profile uses in-memory implementations so the evaluator can run the project immediately after cloning. The Docker profile switches to PostgreSQL, OpenSearch, Redis shards, and Kafka.

2. Dataset Source And Loading Instructions

The selected real dataset is AmazonQAC:

- Source: <https://huggingface.co/datasets/amazon/AmazonQAC>
- License: CDLA-Permissive-2.0
- Relevant fields: `final_search_term`, `popularity`, `search_time`, `prefixes`

The system does not train a machine learning model. It imports real search queries, stores normalized query records, indexes them for prefix lookup, and ranks suggestions using historical and recent counts.

For grading, load at least 100,000 unique AmazonQAC queries. The repository does not commit the full dataset because it is large. The portable smoke profile uses a small seed only so the UI and APIs can be tested immediately; it is not the grading dataset.

The importer accepts the assignment's simple format:

```
query,count
laptop,23222
iphone 15,45222
```

It also accepts AmazonQAC CSV/JSONL fields:

```
final_search_term,popularity,search_time,prefixes
```

Place an AmazonQAC export under `data/`, for example:

```
data/amazonqac_100k.csv
```

Production-like Docker run with PostgreSQL/OpenSearch/Redis/Kafka:

```
TYPEAHEAD_DATASET_IMPORT_PATH=/data/amazonqac_100k.csv \
TYPEAHEAD_DATASET_IMPORT_LIMIT=100000 \
docker compose up --build
```

Portable smoke run without external services:

```
mvn -Dmaven.repo.local=.m2/repository spring-boot:run
```

The smoke run uses a small built-in seed so APIs and UI can be tested on any desktop. The real dataset path above is the intended grading dataset flow.

3. API Documentation

Swagger/OpenAPI is available at `/docs` when the app is running.

API	Purpose
GET <code>/suggest?</code> <code>q=iph&rank=count</code>	Top 10 prefix suggestions sorted by historical count.

GET /suggest? q=iph&rank=hybrid	Top 10 prefix suggestions using recency-aware ranking.
POST /search	Accepts a search query and queues a search event.
GET /trending?window=1h	Returns trending searches for 1h , 24h , or 7d .
GET /cache/debug? prefix=iph	Shows normalized prefix, selected cache node, hit/miss, key, TTL, and ring settings.
GET /metrics	Shows latency, p95, cache hit rate, DB writes, write reduction, and cache-node distribution.

Example search request:

```
curl -X POST 'http://127.0.0.1:8080/search' \
  -H 'Content-Type: application/json' \
  -d '{"query":"iphone 15"}'
```

Example suggestion response:

```
{
  "query": "iph",
  "rank": "hybrid",
  "suggestions": [
    {
      "query": "iphone 15",
      "historical_count": 850000,
      "recent_count_1h": 312,
      "recent_count_24h": 1205,
      "score": 918240
    }
  ],
  "cache": {
    "hit": true,
    "node": "cache-node-2",
    "key": "suggest:v1:1:hybrid:iph"
  },
  "latency_ms": 3.7
}
```

4. Design Choices And Trade-offs

PostgreSQL is the source of truth for query records in the Docker profile. It stores normalized queries, historical counts, last search time, and trend buckets. The trade-off is that PostgreSQL is not used as the hot prefix read path; Redis and OpenSearch handle the latency-sensitive path.

OpenSearch stores the suggestion read model. It supports prefix autocomplete with weighted ranking. This keeps suggestion reads fast, but updates are eventually consistent because search events are batched before weights are refreshed.

Redis is used as a distributed prefix cache. The application includes a Java consistent hash ring with virtual nodes so each prefix cache key maps deterministically to one cache shard. This makes cache-node ownership and remapping visible through /cache/debug .

Kafka is used as the durable search-event buffer in the Docker profile. Search submissions return quickly after publishing an event. Batch workers aggregate repeated searches every 2 seconds or 500 events, reducing database and index writes. The trade-off is brief staleness: PostgreSQL remains durable, while Redis and OpenSearch may lag until the next flush or cache invalidation.

Trending ranking combines historical and recent activity:

```
score =  
  log10(historical_count + 1) * 100000  
  + log10(recent_count_1h + 1) * 60000  
  + log10(recent_count_24h + 1) * 25000
```

Recent searches are stored in 5-minute buckets. Old spikes stop dominating because they fall out of the active time window. Hybrid cache entries also use a shorter TTL than historical ranking.

5. Performance Report

Benchmark command:

```
./scripts/benchmark.sh
```

Measured smoke-profile results:

Metric	Value
Suggest requests	201
Cache hits	184
Cache misses	17
Cache hit rate	91.54%
Average suggest latency	0.91 ms
P95 suggest latency	1.62 ms
Search events received	500
Kafka events published	500
DB write operations	5
Writes saved by batching	495
Write reduction	99.00%
Cache node distribution	cache-node-3: 75, cache-node-2: 126

These benchmark numbers were measured in the portable smoke profile so the project can be verified without a large dataset download. For a final full-dataset run, load the 100,000-query AmazonQAC export through Docker Compose and rerun the same benchmark script.

Demo Evidence

The UI supports a search input, debounced suggestions, keyboard navigation, search submission, ranking toggle, trending panel, cache debug panel, metrics panel, loading states, and error states.

Screenshots are included in:

- docs/screenshots/search-ui-overall-mode.png
- docs/screenshots/search-ui-suggestions.png